Technical Specifications, Architecture, and System Design

Student Name

Mubarakov Islam

Group

11-314a

Assignment

1st task series (2nd semester)

GitHub Repository

https://github.com/iz-mv/Kopilkin

Task Block 1 Scope

High-level Scope

1. Technical specifications / Requirements
2. User Story + Use Cases + NFR (text)
3. C4 (L1-L3) + Sequence Diagram
4. Microservices. 3 in total at that moment

1. Technical Specifications / Requirements

1. Техническая спецификация / Требования

RU

1.1. Название системы

Kopilkin

1.2. Тип системы

Kopilkin — это mobile-first веб-приложение для управления личными накоплениями, анализа расходов и поддержки накопительных целей. Система предоставляет пользователю возможность регистрироваться, входить в аккаунт, учитывать транзакции, анализировать траты по категориям и отслеживать прогресс по финансовым целям.

1.3. Основная цель системы

Основная цель Kopilkin — помочь пользователю лучше понимать свои финансовые привычки, контролировать повседневные расходы и формировать накопления с помощью простых цифровых инструментов.

1.4. Архитектурный подход

Система проектируется как **микросервисное web application**, состоящее как минимум из трёх независимых сервисов. Такой подход выбран потому, что задание прямо требует минимум 3 микросервиса, а сам курс ориентирован на проектирование distributed systems и дальнейшее развитие архитектуры в следующих task series.

1.5. Базовые модули системы

На первом этапе система состоит из следующих основных сервисов:

- **Auth Service**
- **Transaction Service**
- **Savings Goal Service**

Дополнительно в архитектуре могут быть предусмотрены:

- **User Profile Service**

- **Analytics Service**
- **Notification Service**
- **Assistant Service**

1.6. Поддерживаемые способы аутентификации

Система должна поддерживать:

- регистрацию через email и password,
- вход через email и password,
- вход через **Google account** с использованием OAuth 2.0.

1.7. Основные функции системы

Система должна обеспечивать следующие функции:

1. Регистрация пользователя.
2. Аутентификация пользователя.
3. Вход через Google account.
4. Создание и просмотр транзакций.
5. Категоризация расходов.
6. Создание накопительных целей.
7. Просмотр прогресса накопления.
8. Отображение краткой финансовой сводки.
9. Подготовка к интеграции с внешним transaction provider.

1.8. Формат клиентской части

Клиентская часть реализуется как **mobile-first web interface**, оптимизированный под использование на смартфоне. Полноценный нативный iOS/Android клиент в текущий scope не входит.

1.9. Внешние интеграции

На первом этапе допускаются следующие внешние интеграции:

- **Google OAuth** для входа,
- mock/external transaction provider для демонстрации получения транзакционных данных.

1.10. Хранение данных

Для архитектурной версии проекта основным вариантом хранения данных рассматривается **PostgreSQL**. Для локального демо допускается

использование упрощённого хранилища, если это потребуется для ускорения разработки. В рамках следующих task blocks архитектура может быть расширена за счёт дополнительных data components и caching layer. Это соответствует структуре курса, где отдельно выделены data layer и Redis-like caching для следующего этапа.

EN

1.1. System Name

Kopilkin

1.2. System Type

Kopilkin is a mobile-first web application for personal savings management, spending analytics, and support for savings goals. The system allows a user to register, log in, track transactions, analyze spending by category, and monitor progress toward financial goals.

1.3. Main Goal of the System

The main goal of Kopilkin is to help users better understand their financial habits, control everyday spending, and build savings through simple digital tools.

1.4. Architectural Approach

The system is designed as a **microservice-based web application** consisting of at least three independent services. This approach is chosen because the assignment explicitly requires at least 3 microservices, while the course itself is focused on distributed systems design and further architectural development in later task series.

1.5. Core System Modules

At the first stage, the system consists of the following main services:

- **Auth Service**
- **Transaction Service**
- **Savings Goal Service**

Additionally, the architecture may later include:

- **User Profile Service**

- **Analytics Service**
- **Notification Service**
- **Assistant Service**

1.6. Supported Authentication Methods

The system should support:

- registration with email and password,
- login with email and password,
- sign-in with a **Google account** using OAuth 2.0.

1.7. Main System Functions

The system shall provide the following functions:

1. User registration.
2. User authentication.
3. Google account sign-in.
4. Transaction creation and viewing.
5. Expense categorization.
6. Savings goal creation.
7. Savings progress tracking.
8. Display of a short financial summary.
9. Preparation for integration with an external transaction provider.

1.8. Client Format

The client application is implemented as a **mobile-first web interface** optimized for smartphone usage. A fully native iOS/Android client is outside the current scope.

1.9. External Integrations

At the first stage, the following external integrations are allowed:

- **Google OAuth** for authentication,
- a mock/external transaction provider for demonstrating transaction data ingestion.

1.10. Data Storage

For the architectural version of the project, **PostgreSQL** is considered the primary data storage option. For local demo purposes, a simplified storage

approach may be used if needed to speed up development. In later task blocks, the architecture can be extended with additional data components and a caching layer, which aligns with the course structure where the data layer and Redis-like caching are addressed separately in the next stage.

Functional Requirements / Функциональные требования

RU

1.11. Аутентификация и доступ

FR-1. Система должна позволять пользователю зарегистрироваться по email и password.
FR-2. Система должна позволять пользователю войти в систему по email и password.
FR-3. Система должна позволять пользователю войти через Google account.
FR-4. Система должна создавать и возвращать токен доступа после успешной аутентификации.
FR-5. Система должна определять текущего аутентифицированного пользователя.

1.12. Транзакции

FR-6. Система должна позволять пользователю добавлять транзакцию вручную.
FR-7. Каждая транзакция должна содержать сумму, дату, категорию и опционально описание.
FR-8. Система должна позволять пользователю просматривать список своих транзакций.
FR-9. Система должна группировать или фильтровать транзакции по категориям.

1.13. Цели накопления

FR-10. Система должна позволять пользователю создавать накопительную цель.
FR-11. Каждая цель должна содержать название, целевую сумму и текущий прогресс.
FR-12. Система должна отображать прогресс по каждой накопительной цели.

FR-13. Система должна позволять пользователю просматривать список своих накопительных целей.

1.14. Сводка

FR-14. Система должна показывать краткую сводку по расходам пользователя.
FR-15. Система должна показывать сумму расходов по категориям за выбранный период.
FR-16. Система должна отображать связь между расходами и целями накопления в упрощённом виде.

1.15. Подготовка к интеграции

FR-17. Система должна поддерживать архитектурную возможность подключения внешнего transaction provider.
FR-18. Система должна предусматривать расширение для рекомендаций или AI-based assistant в будущем.

EN

1.11. Authentication and Access

FR-1. The system shall allow a user to register with email and password.
FR-2. The system shall allow a user to log in with email and password.
FR-3. The system shall allow a user to sign in with a Google account.
FR-4. The system shall create and return an access token after successful authentication.
FR-5. The system shall identify the current authenticated user.

1.12. Transactions

FR-6. The system shall allow a user to add a transaction manually.
FR-7. Each transaction shall contain amount, date, category, and description.
FR-8. The system shall allow a user to view their list of transactions.
FR-9. The system shall group or filter transactions by category.

1.13. Savings Goals

FR-10. The system shall allow a user to create a savings goal.
FR-11. Each goal shall contain a title, target amount, and current progress.

FR-12. The system shall display progress for each savings goal.
FR-13. The system shall allow a user to view their list of savings goals.

1.14. Summary

FR-14. The system shall display a short spending summary for the user.
FR-15. The system shall show spending totals by category for a selected period.
FR-16. The system shall display a simplified relationship between spending and savings goals.

1.15. Integration Readiness

FR-17. The system shall support the architectural possibility of connecting to an external transaction provider.
FR-18. The system shall provide an extension point for future recommendations or an AI-based assistant.

2. User Stories + Use Cases + NFR

2. Пользовательские истории + варианты использования + нефункциональные требования

2.1 User Stories / Пользовательские истории

RU

Ниже приведены основные user stories для **Kopilkin**.

US-1. Регистрация через email
Как новый пользователь, я хочу зарегистрироваться по email и password, чтобы получить доступ к личному кабинету и начать пользоваться системой.

US-2. Вход через email
Как зарегистрированный пользователь, я хочу входить по email и password, чтобы безопасно получать доступ к своим данным.

US-3. Вход через Google
Как пользователь, я хочу войти через Google account, чтобы не создавать отдельный пароль и быстрее начать использовать системы.

US-4. Просмотр своего профиля
Как пользователь, я хочу видеть данные своего профиля, чтобы понимать, под каким аккаунтом я работаю и какие настройки у меня сохранены.

US-5. Добавление транзакции вручную
Как пользователь, я хочу вручную добавить транзакцию, чтобы учитывать свои расходы даже без автоматической банковской интеграции.

US-6. Категоризация расходов
Как пользователь, я хочу присваивать транзакциям категории, чтобы понимать, на что именно уходят мои деньги.

US-7. Просмотр списка транзакций
Как пользователь, я хочу видеть список своих транзакций, чтобы отслеживать историю расходов.

US-8. Просмотр расходов по категориям
Как пользователь, я хочу видеть суммы по категориям, чтобы анализировать своё финансовое поведение.

US-9. Создание цели накопления
Как пользователь, я хочу создать цель накопления, чтобы копить деньги на конкретную задачу или покупку.

US-10. Отслеживание прогресса по цели
Как пользователь, я хочу видеть прогресс накопления по каждой цели, чтобы сохранять мотивацию и понимать, сколько ещё осталось.

US-11. Получение краткой финансовой сводки
Как пользователь, я хочу видеть краткую сводку по расходам и накоплениям, чтобы быстро понимать текущее финансовое состояние.

US-12. Использование внешнего источника транзакций
Как пользователь, я хочу в будущем подключить внешний источник транзакций, чтобы сократить ручной ввод данных.

US-13. Получение рекомендаций по накоплению
Как пользователь, я хочу получать простые рекомендации по накоплению, чтобы лучше планировать свои деньги.

US-14. Получение напоминаний
Как пользователь, я хочу получать напоминания о целях и накоплениях, чтобы не забывать поддерживать финансовую дисциплину.

EN

Below are the main user stories for the **Kopilkin** system.

US-1. Email registration
As a new user, I want to register with email and password so that I can access my personal account and start using the system.

US-2. Email login
As a registered user, I want to log in with email and password so that I can securely access my data.

US-3. Google sign-in
As a user, I want to sign in with my Google account so that I do not need to create a separate password and can start using the app faster.

US-4. View my profile
As a user, I want to see my profile data so that I know which account I am using and what settings are saved.

US-5. Add a transaction manually
As a user, I want to add a transaction manually so that I can track my spending even without automatic banking integration.

US-6. Categorize expenses
As a user, I want to assign categories to transactions so that I can understand where my money goes.

US-7. View the transaction list
As a user, I want to see the list of my transactions so that I can track my spending history.

US-8. View spending by category
As a user, I want to see totals by category so that I can analyze my financial behavior.

US-9. Create a savings goal
As a user, I want to create a savings goal so that I can save money for a specific purpose or purchase.

US-10. Track goal progress
As a user, I want to see the progress of each savings goal so that I can stay motivated and understand how much is left.

US-11. Receive a short financial summary
As a user, I want to see a short summary of my spending and savings so that I can quickly understand my current financial situation.

US-12. Use an external transaction source
As a user, I want to connect an external transaction source in the future so that I can reduce manual data entry.

US-13. Receive savings recommendations
As a user, I want to receive simple savings recommendations so that I can plan my money better.

US-14. Receive reminders
As a user, I want to receive reminders about my goals and savings so that I do not forget to maintain financial discipline.

2.2 Use Cases / Варианты использования

RU

Ниже перечислены ключевые use cases системы.

UC-1. Регистрация пользователя

Актор: User
Предусловие: пользователь ещё не зарегистрирован.
Основной сценарий:

1. Пользователь открывает форму регистрации.
2. Вводит email и password.
3. Система проверяет корректность данных.
4. Система создаёт учётную запись.
5. Пользователь получает подтверждение успешной регистрации.
   Результат: создан новый аккаунт пользователя.

UC-2. Вход пользователя по email/password

Актор: User
Предусловие: пользователь уже зарегистрирован.
Основной сценарий:

1. Пользователь открывает форму входа.
2. Вводит email и password.
3. Система проверяет данные.
4. Система создаёт access token / session.
5. Пользователь попадает в систему.
   Результат: пользователь аутентифицирован.

UC-3. Вход через Google account

Актор: User
Предусловие: у пользователя есть Google account.
Основной сценарий:

1. Пользователь нажимает “Continue with Google”.
2. Система перенаправляет его к Google OAuth provider.
3. Пользователь подтверждает вход.
4. Система получает данные аутентификации от Google.
5. Система создаёт или находит локального пользователя.

6. Система выдаёт access token / session.
   Результат: пользователь аутентифицирован через внешний identity provider.

UC-4. Добавление транзакции

Актор: User
Предусловие: пользователь вошёл в систему.
Основной сценарий:

1. Пользователь открывает экран транзакций.
2. Нажимает “Add transaction”.
3. Указывает сумму, дату, категорию и описание.
4. Система сохраняет транзакцию.
5. Обновляется список транзакций и краткая сводка.
   Результат: новая транзакция добавлена.

UC-5. Просмотр транзакций

Актор: User
Предусловие: пользователь вошёл в систему.
Основной сценарий:

1. Пользователь открывает список транзакций.
2. Система запрашивает данные из Transaction Service.
3. Система отображает список операций.
   Результат: пользователь видит историю расходов.

UC-6. Просмотр аналитики по категориям

Актор: User
Предусловие: у пользователя есть хотя бы одна транзакция.
Основной сценарий:

1. Пользователь открывает экран аналитики или dashboard.
2. Система агрегирует траты по категориям.
3. Система показывает суммы и простые визуальные сводки.
   Результат: пользователь видит структуру своих расходов.

UC-7. Создание накопительной цели

Актор: User
Предусловие: пользователь вошёл в систему.
Основной сценарий:

1. Пользователь открывает раздел целей.
2. Нажимает “Create goal”.
3. Указывает название цели и целевую сумму.
4. Система создаёт новую цель накопления.
5. Цель появляется в списке.
   Результат: новая накопительная цель создана.

UC-8. Просмотр прогресса накопления

Актор: User
Предусловие: у пользователя есть хотя бы одна цель накопления.
Основной сценарий:

1. Пользователь открывает экран целей.
2. Система получает информацию о целях и прогрессе.
3. Система отображает текущее состояние по каждой цели.
   Результат: пользователь видит прогресс накопления.

UC-9. Получение краткой финансовой сводки

Актор: User
Предусловие: пользователь вошёл в систему.
Основной сценарий:

1. Пользователь открывает главный экран.
2. Система получает краткие данные по расходам и целям.
3. Система отображает summary dashboard.
   Результат: пользователь быстро видит основные показатели.

UC-10. Подключение внешнего transaction source

Актор: User / External Transaction Provider
Предусловие: в системе предусмотрена интеграция с внешним провайдером.
Основной сценарий:

1. Пользователь инициирует подключение внешнего источника (банковского API).
2. Система проходит процедуру авторизации / настройки интеграции.
3. Внешний provider передаёт транзакционные данные.
4. Система сохраняет или обновляет операции пользователя.
   Результат: транзакции могут поступать из внешнего источника.

EN

Below are the key use cases of the system.

UC-1. User registration

Actor: User
Precondition: the user is not yet registered.
Main flow:

1. The user opens the registration form.
2. Enters email and password.
3. The system validates the input data.
4. The system creates an account.
5. The user receives confirmation of successful registration.
   Result: a new user account is created.

UC-2. Email/password login

Actor: User
Precondition: the user is already registered.
Main flow:

1. The user opens the login form.
2. Enters email and password.
3. The system validates the credentials.
4. The system creates an access token / session.
5. The user enters the system.
   Result: the user is authenticated.

UC-3. Google account sign-in

Actor: User
Precondition: the user has a Google account.
Main flow:

1. The user clicks “Continue with Google”.
2. The system redirects the user to the Google OAuth provider.
3. The user confirms sign-in.
4. The system receives authentication data from Google.
5. The system creates or finds the local user account.
6. The system issues an access token / session.
   Result: the user is authenticated through an external identity provider.

UC-4. Add a transaction

Actor: User
Precondition: the user is logged in.
Main flow:

1. The user opens the transactions screen.
2. Clicks “Add transaction”.
3. Enters amount, date, category, and description.
4. The system stores the transaction.
5. The transaction list and short summary are updated.
   Result: a new transaction is added.

UC-5. View transactions

Actor: User
Precondition: the user is logged in.
Main flow:

1. The user opens the transaction list.
2. The system requests data from the Transaction Service.
3. The system displays the operation list.
   Result: the user sees spending history.

UC-6. View spending analytics by category

Actor: User
Precondition: the user has at least one transaction.
Main flow:

1. The user opens the analytics screen or dashboard.
2. The system aggregates spending by category.
3. The system displays totals and simple visual summaries.
   Result: the user sees the structure of their spending.

UC-7. Create a savings goal

Actor: User
Precondition: the user is logged in.
Main flow:

1. The user opens the goals section.
2. Clicks “Create goal”.
3. Enters the goal title and target amount.

4. The system creates a new savings goal.
5. The goal appears in the list.
   Result: a new savings goal is created.

UC-8. View savings progress

Actor: User
Precondition: the user has at least one savings goal.
Main flow:

1. The user opens the goals screen.
2. The system retrieves goal and progress information.
3. The system displays the current state of each goal.
   Result: the user sees savings progress.

UC-9. Receive a short financial summary

Actor: User
Precondition: the user is logged in.
Main flow:

1. The user opens the main screen.
2. The system fetches short spending and goal-related data.
3. The system displays a summary dashboard.
   Result: the user quickly sees the key financial indicators.

UC-10. Connect an external transaction source

Actor: User / External Transaction Provider
Precondition: the system supports integration with an external provider.
Main flow:

1. The user initiates connection of an external source (banking API).
2. The system performs authorization / integration setup.
3. The external provider sends transaction data.
4. The system stores or updates the user's operations.
   Result: transactions can be received from an external source.

2.3 NFR — Non-Functional Requirements

2.3 Нефункциональные требования

RU

Ниже указаны основные нефункциональные требования для **Kopilkin**.

NFR-1. Usability / Удобство использования
Система должна быть спроектирована в формате **mobile-first web application**, чтобы пользователь мог комфортно работать со смартфона. Интерфейс должен быть простым, понятным и не перегруженным.

NFR-2. Performance / Производительность
Основные пользовательские действия, такие как вход, просмотр транзакций и открытие целей, должны выполняться с минимальной задержкой при нормальной нагрузке.

NFR-3. Availability / Доступность
Система должна быть доступна для использования в стандартном режиме, за исключением периодов технического обслуживания или сбоев внешних интеграций.

NFR-4. Scalability / Масштабируемость
Архитектура должна поддерживать расширение системы за счёт независимого масштабирования сервисов. Это особенно важно, поскольку курс ориентирован на проектирование distributed systems и развитие системы в следующих блоках.

NFR-5. Security / Безопасность
Система должна обеспечивать безопасную аутентификацию пользователя, защиту access token и базовую защиту пользовательских данных.
Вход через Google должен использовать стандартный OAuth 2.0 flow.

NFR-6. Maintainability / Поддерживаемость
Система должна быть разделена на отдельные сервисы с понятными зонами ответственности, чтобы упростить сопровождение, тестирование и доработку.

NFR-7. Extensibility / Расширяемость
Архитектура должна позволять в будущем подключить:

- external transaction providers,
- analytics improvements,

- assistant/AI module,
- Kafka-based communication,
- caching layer,
- routing layer.

Это согласуется с продолжением задания в Task Block 3.

NFR-8. Fault Isolation / Изоляция отказов
Сбой одного сервиса не должен полностью выводить из строя всю систему. Например, проблемы во внешней transaction integration не должны полностью блокировать вход пользователя.

NFR-9. Data Consistency / Согласованность данных
Данные о пользователях, транзакциях и целях должны храниться последовательно и логично, без некорректных связей между сущностями.

NFR-10. Deployability / Развёртываемость
Система должна быть пригодна для локального запуска и демонстрации, а также для дальнейшего переноса в контейнеризированную или облачную среду.

NFR-11. Interoperability / Совместимость интеграций
Система должна быть готова к взаимодействию с внешними сервисами через API, поскольку integration methods и user identification входят в тематику курса.

NFR-12. Observability readiness / Готовность к наблюдаемости
На текущем этапе полноценная observability может не быть реализована, но архитектура должна допускать будущее добавление логирования, метрик и трассировки.

EN

Below are the main non-functional requirements for **Kopilkin**.

NFR-1. Usability
The system shall be designed as a **mobile-first web application** so that the user can comfortably interact with it from a smartphone. The interface shall be simple, clear, and not overloaded.

NFR-2. Performance
Core user actions such as login, viewing transactions, and opening savings goals shall be performed with minimal delay under normal load.

NFR-3. Availability
The system shall remain available for standard use except during maintenance periods or failures of external integrations.

NFR-4. Scalability
The architecture shall support system growth through independent scaling of services. This is especially important because the course focuses on distributed systems and further architectural evolution in later task blocks.

NFR-5. Security
The system shall provide secure user authentication, protection of access tokens, and basic protection of user data.
Google sign-in shall use a standard OAuth 2.0 flow.

NFR-6. Maintainability
The system shall be divided into separate services with clear responsibilities in order to simplify maintenance, testing, and future changes.

NFR-7. Extensibility
The architecture shall allow future connection of:

- external transaction providers,
- improved analytics,
- an assistant/AI module,
- Kafka-based communication,
- a caching layer,
- a routing layer.

This is aligned with the continuation of the assignment in Task Block 3.

NFR-8. Fault Isolation
Failure of one service shall not completely disable the entire system. For example, issues in the external transaction integration shall not fully block local user login.

NFR-9. Data Consistency
User, transaction, and savings goal data shall be stored in a consistent and logically connected way without invalid relationships between entities.

NFR-10. Deployability
The system shall be suitable for local execution and demonstration, as well as for future migration to a containerized or cloud environment.

NFR-11. Interoperability
The system shall be ready to interact with external services via APIs, since integration methods and user identification are part of the course topics.

NFR-12. Observability readiness
At the current stage, full observability may not yet be implemented, but the architecture shall allow future addition of logging, metrics, and tracing.

2.4 Short concluding paragraph for the report / Короткий завершающий абзац для отчёта

RU

Представленные user stories, use cases и NFR показывают, что Kopilkin проектируется не просто как интерфейс для ручного ввода расходов, а как расширяемая цифровая платформа для управления накоплениями и анализа финансового поведения. Такой подход позволяет обосновать выбор микросервисной архитектуры, а также подготовить систему к дальнейшему развитию на следующих этапах.

EN

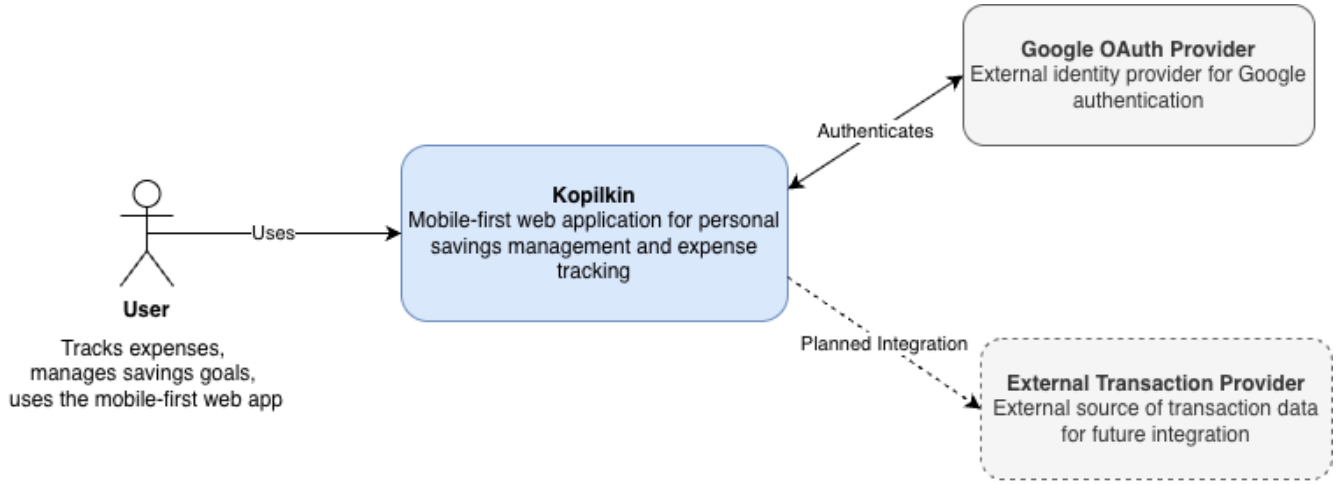
The presented user stories, use cases, and NFR demonstrate that Kopilkin is designed not merely as an interface for manual expense entry, but as an extensible digital platform for savings management and financial behavior analysis. This approach helps justify the choice of a microservice architecture and prepares the system for further development in later stages.

3. C4 (L1-3) + Sequence Diagram

3.1 C4 Level 1 — System Context Diagram

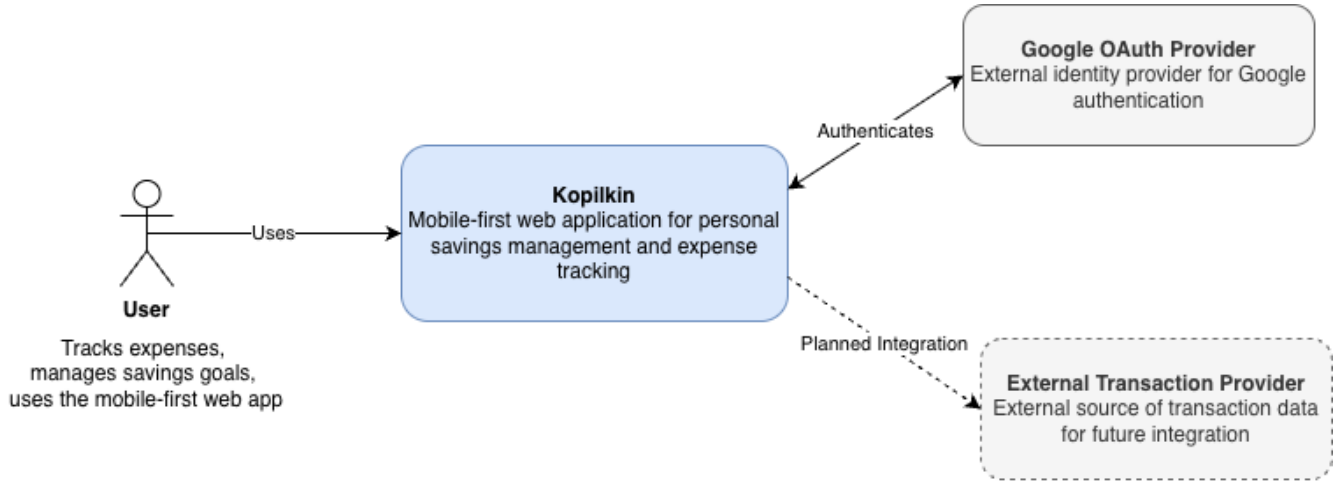
RU

C4 Level 1 показывает Kopilkin как единую программную систему в её внешнем окружении. На данном уровне отображаются только пользователь, сама система и внешние интеграции, без детализации внутренних сервисов. Пользователь взаимодействует с Kopilkin через mobile-first web interface. Для аутентификации предусмотрена интеграция с Google OAuth provider, а для дальнейшего развития архитектуры — подключение внешнего transaction provider. Такой контекст соответствует заявленным функциональным требованиям и поддерживает будущую расширяемость системы.



EN

C4 Level 1 presents Kopilkin as a single software system within its external environment. At this level, only the user, the system itself, and external integrations are shown, without internal service decomposition. The user interacts with Kopilkin through a mobile-first web interface. The architecture includes Google OAuth integration for authentication and a planned external transaction provider for future transaction import. This context matches the stated functional requirements and supports future system extensibility.



3.2 C4 Level 2 — Container Diagram

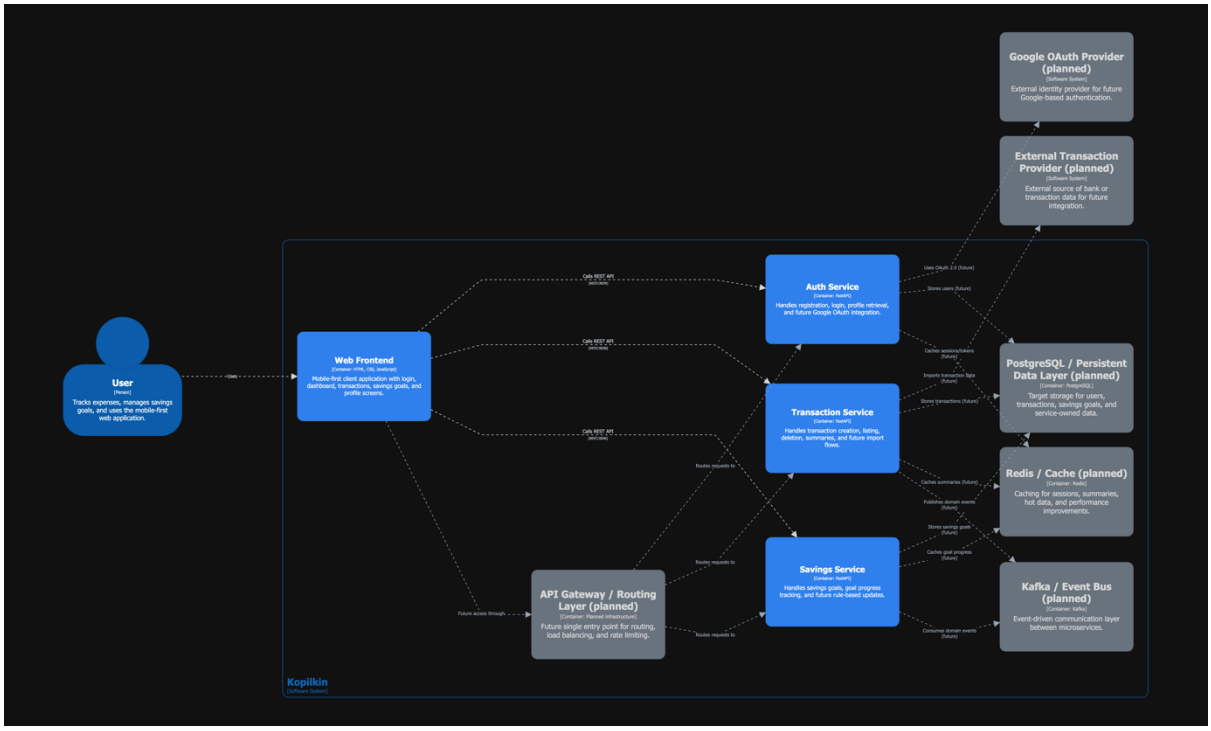
RU

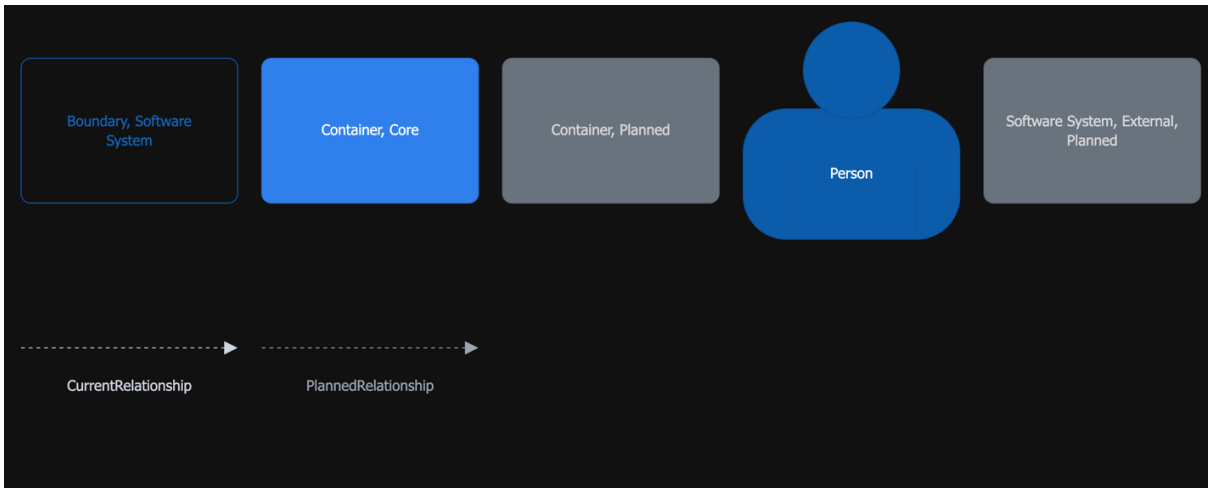
Диаграмма **C4 Level 2** показывает внутреннюю высокоуровневую структуру системы **Kopilkin**. На этом уровне система уже рассматривается не как один общий блок, а как набор крупных архитектурных частей и их связей.

В текущем варианте ядро системы состоит из **Web Frontend**, **Auth Service**, **Transaction Service** и **Savings Service**.
Web Frontend отвечает за пользовательский интерфейс и взаимодействие пользователя с системой.
Auth Service реализует регистрацию, вход и получение данных пользователя.
Transaction Service отвечает за создание, просмотр и удаление транзакций, а также за формирование сводки по доходам и расходам.
Savings Service управляет целями накопления и отслеживает прогресс их выполнения.

Кроме текущих компонентов, диаграмма также показывает запланированные элементы будущей архитектуры: **API Gateway / Routing Layer**, **Kafka / Event Bus**, **PostgreSQL / Persistent Data Layer**, **Redis / Cache**, а также внешние интеграции с **Google OAuth Provider** и **External Transaction Provider**. Эти элементы обозначены как **planned**, поскольку они относятся к следующим этапам развития проекта.

Таким образом, диаграмма отражает как текущую MVP-структуру системы, так и её целевое архитектурное развитие.





EN

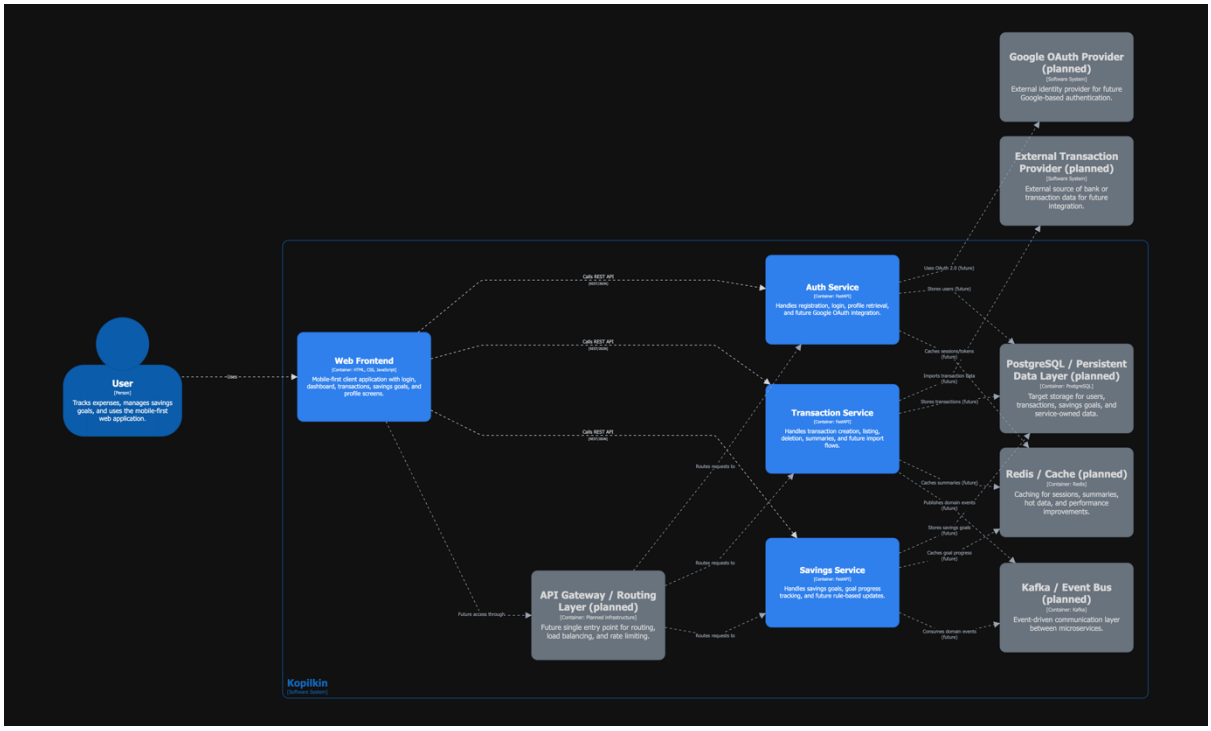
The **C4 Level 2** diagram shows the high-level internal structure of the **Kopilkin** system. At this level, the system is no longer presented as a single block, but as a set of major architectural parts and their relationships.

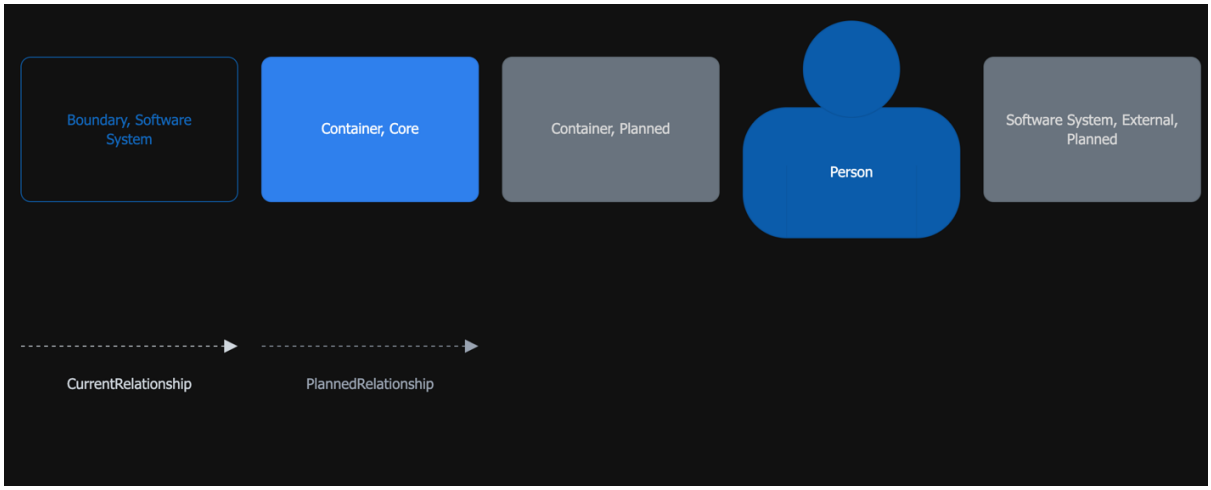
In the current version, the core of the system consists of **Web Frontend**, **Auth Service**, **Transaction Service**, and **Savings Service**.
The **Web Frontend** provides the user interface and user interaction with the system.
The **Auth Service** handles registration, login, and user profile retrieval.

The **Transaction Service** is responsible for creating, viewing, and deleting transactions, as well as generating income and expense summaries.
The **Savings Service** manages savings goals and tracks their progress.

In addition to the current components, the diagram also includes planned elements of the future architecture: **API Gateway / Routing Layer**, **Kafka / Event Bus**, **PostgreSQL / Persistent Data Layer**, **Redis / Cache**, as well as external integrations with **Google OAuth Provider** and **External Transaction Provider**. These elements are marked as **planned** because they belong to the next stages of project development.

Therefore, the diagram reflects both the current MVP structure of the system and its target architectural evolution.





3.3 C4 Level 3 — Component Diagram

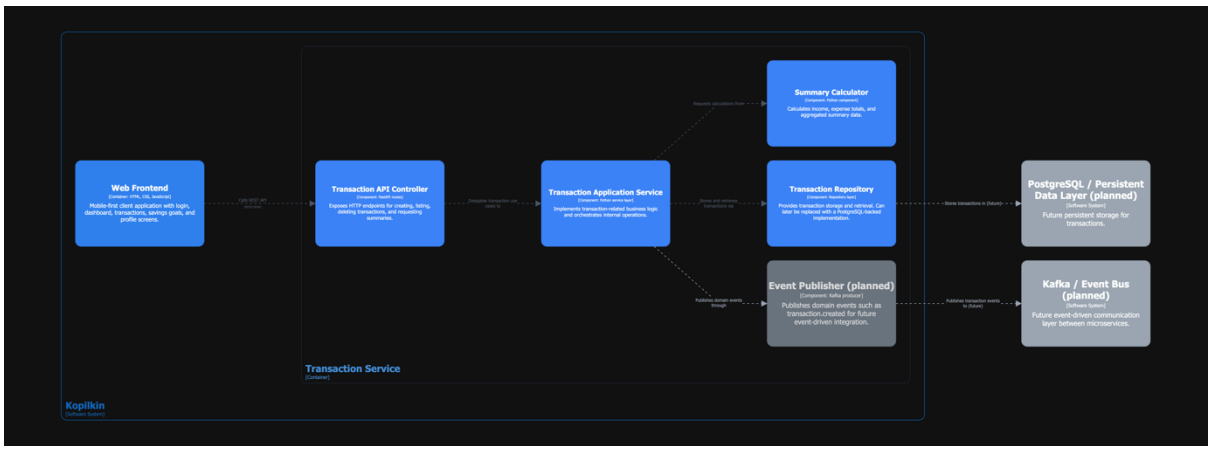
RU

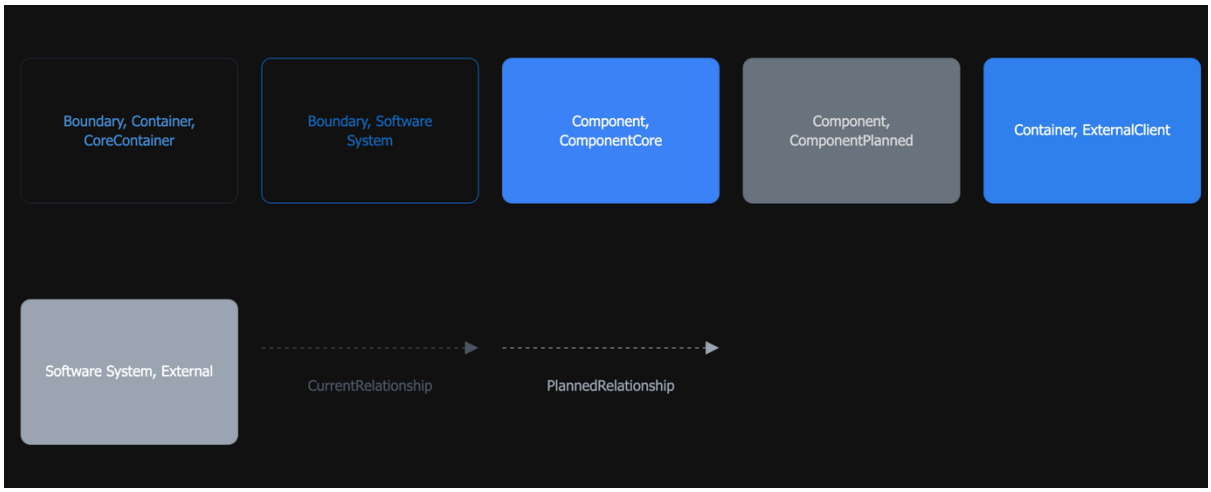
Диаграмма **C4 Level 3** показывает внутреннюю структуру контейнера **Transaction Service**. В отличие от **C4 Level 2**, где отображаются только крупные части системы, уровень **L3** раскрывает логическое разбиение одного сервиса на внутренние компоненты и их взаимодействия.

Внутри **Transaction Service** выделены основные компоненты: **Transaction API Controller**, **Transaction Application Service**, **Summary Calculator** и **Transaction Repository**.
Transaction API Controller принимает HTTP-запросы от клиентской части системы.
Transaction Application Service содержит основную бизнес-логику и координирует выполнение операций.
Summary Calculator отвечает за расчёт агрегированных данных по доходам и расходам.
Transaction Repository представляет слой доступа к данным и отвечает за хранение и получение транзакций.

Кроме того, на диаграмме показан компонент **Event Publisher**, отмеченный как **planned**. В будущем он может использоваться для публикации событий, например transaction.created, в **Kafka** в рамках событийно-ориентированной архитектуры. Также на диаграмме отражён planned-переход к использованию **PostgreSQL** в качестве постоянного хранилища данных.

Текущая реализация сервиса упрощена и часть логики пока находится в одном FastAPI-модуле, однако L3-диаграмма показывает логическое разбиение Transaction Service на внутренние компоненты и отражает целевую архитектуру сервиса с учётом дальнейшего расширения.





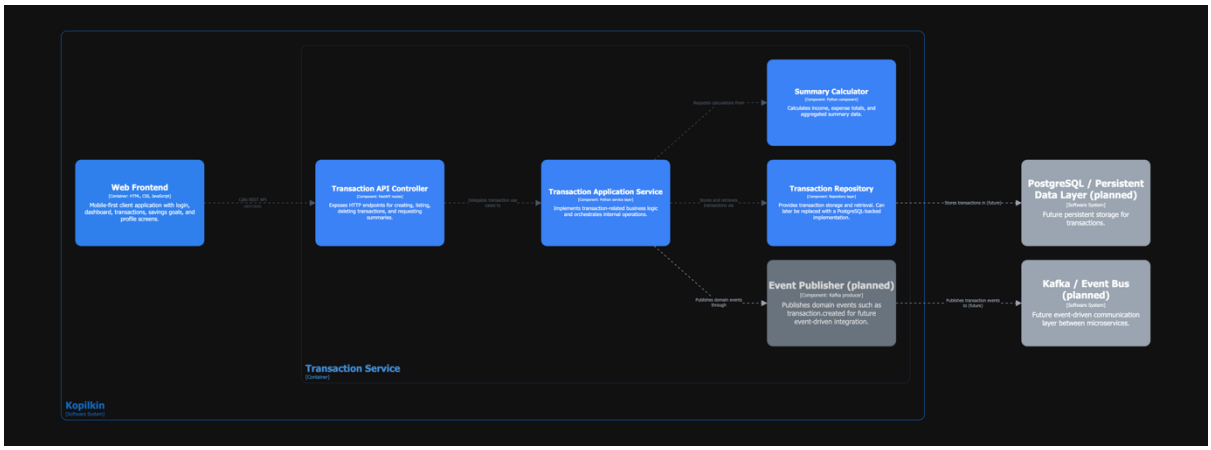
EN

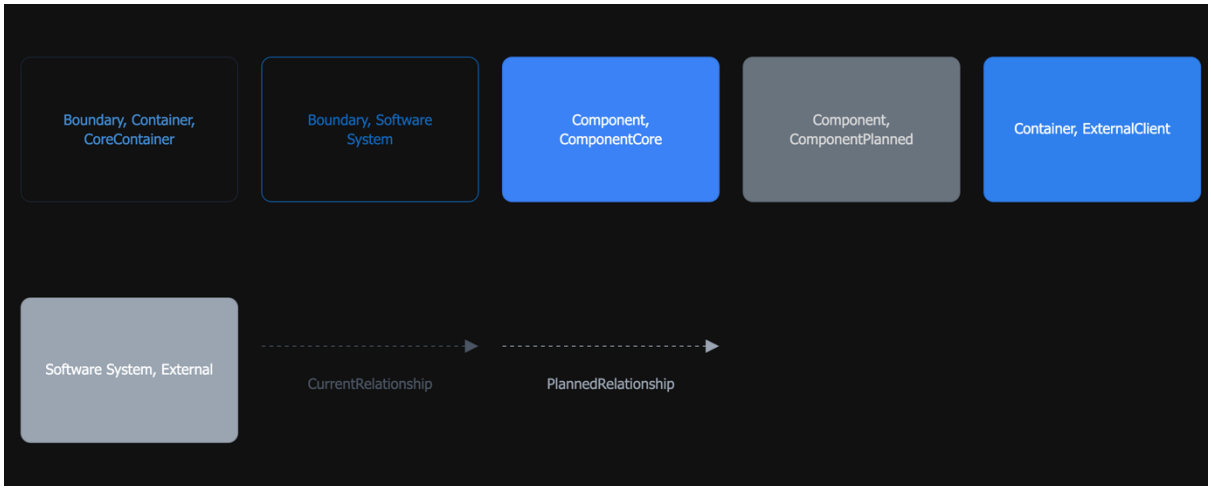
The **C4 Level 3** diagram shows the internal structure of the **Transaction Service** container. Unlike **C4 Level 2**, which presents only the major parts of the system, **Level 3** reveals the logical decomposition of a single service into internal components and their interactions.

Inside the **Transaction Service**, the main components are **Transaction API Controller**, **Transaction Application Service**, **Summary Calculator**, and **Transaction Repository**.
The **Transaction API Controller** accepts HTTP requests from the client side of the system.
The **Transaction Application Service** contains the main business logic and coordinates operations.
The **Summary Calculator** is responsible for computing aggregated income and expense data.
The **Transaction Repository** represents the data access layer and is responsible for storing and retrieving transactions.

In addition, the diagram includes the **Event Publisher** component marked as **planned**. In the future, it may be used to publish events such as transaction.created to **Kafka** as part of an event-driven architecture. The diagram also reflects the planned transition to **PostgreSQL** as a persistent data storage layer.

The current implementation is simplified and some responsibilities are still located in a single FastAPI module, but the L3 diagram shows the logical internal decomposition of the Transaction Service and reflects its intended architecture with future extensibility in mind.





3.4 Sequence Diagram

RU

Диаграмма последовательностей показывает сценарий входа пользователя в систему и добавления новой транзакции. Сначала пользователь открывает страницу входа и вводит свои данные, после чего **Web Frontend** отправляет запрос в **Auth Service**. В ответ сервис аутентификации возвращает токен доступа и данные пользователя, что завершает процесс входа в систему.

После успешной аутентификации пользователь добавляет новую транзакцию через интерфейс приложения. **Web Frontend** отправляет данные транзакции в **Transaction Service**, который обрабатывает запрос и создаёт новую запись. Затем клиентская часть повторно запрашивает список транзакций пользователя и обновлённую сводку по доходам и расходам, чтобы отобразить актуальное состояние данных на экране.

Диаграмма также включает planned-элемент будущей архитектуры: после создания транзакции **Transaction Service** сможет публиковать событие transaction.created в **Kafka / Event Bus**. Это позволит в дальнейшем реализовать событийно-ориентированное взаимодействие между сервисами.

EN

The sequence diagram shows the scenario of user login and adding a new transaction. First, the user opens the login page and enters credentials, after which the **Web Frontend** sends a request to the **Auth Service**. In response, the authentication service returns an access token and user data, which completes the login process.

After successful authentication, the user submits a new transaction through the application interface. The **Web Frontend** sends the transaction data to the **Transaction Service**, which processes the request and creates a new record. Then the client side requests the updated transaction list and the updated income/expense summary in order to display the current state of the data.

The diagram also includes a planned element of the future architecture: after a transaction is created, the **Transaction Service** will be able to publish the transaction.created event to **Kafka / Event Bus**. This will support event-driven communication between services in later stages of development.

4. Microservices (3)

4. Микросервисы (3)

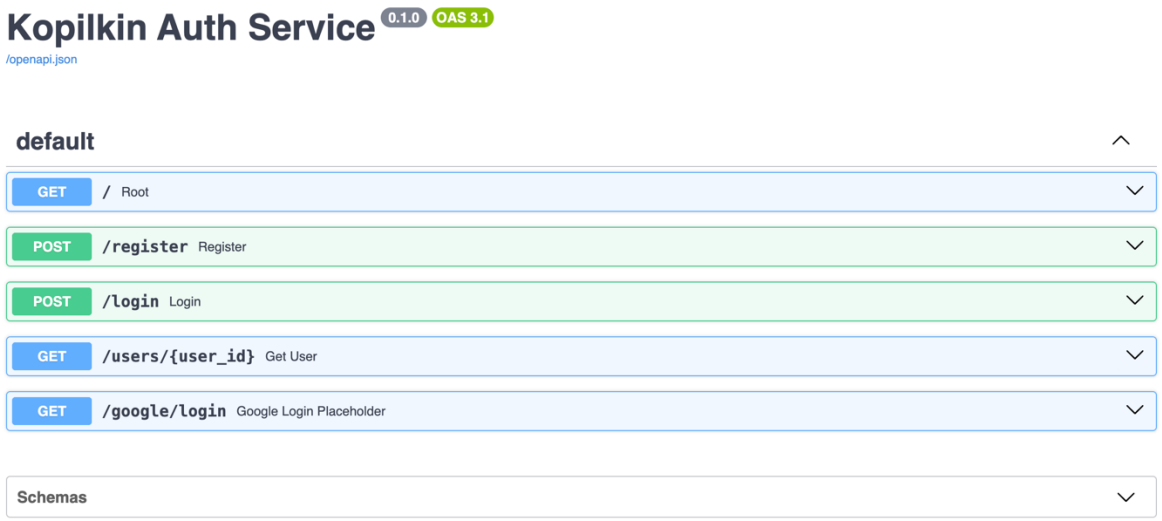
RU

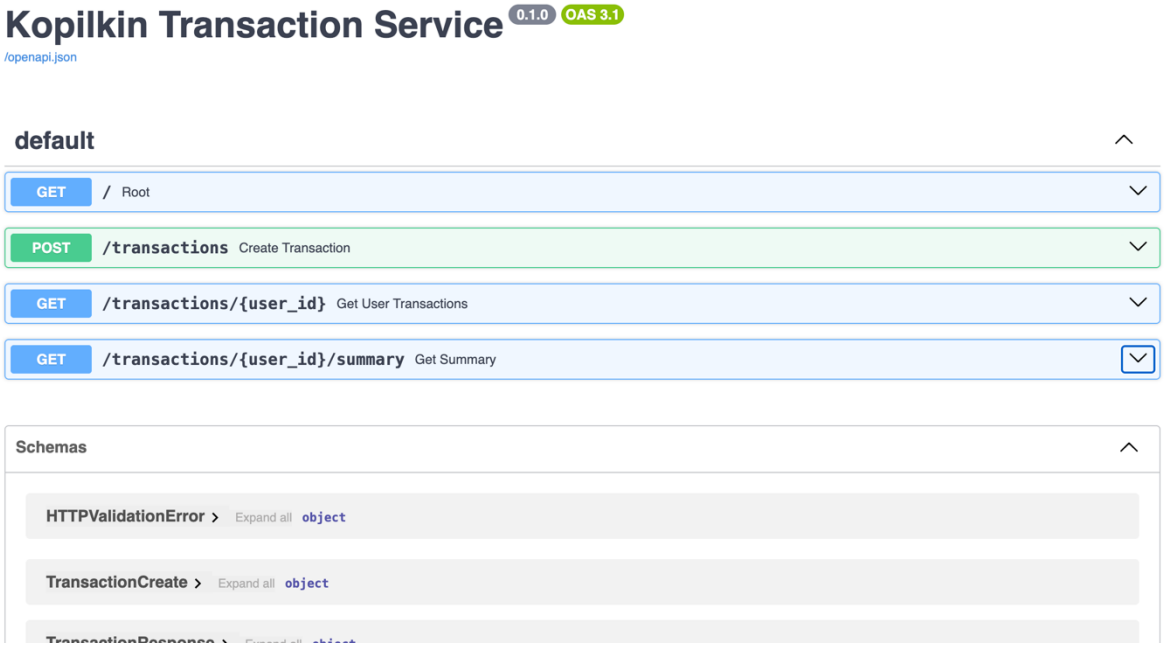
Описание микросервисной архитектуры

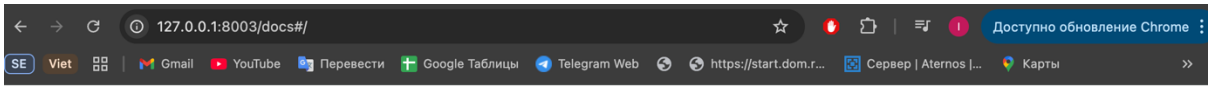
Для первой версии проекта **Kopilkin** была выбрана микросервисная архитектура.
Такой подход соответствует требованиям задания, в котором необходимо предусмотреть не менее трёх микросервисов, а также позволяет логически разделить систему на независимые функциональные части. В текущем MVP были реализованы три базовых сервиса:

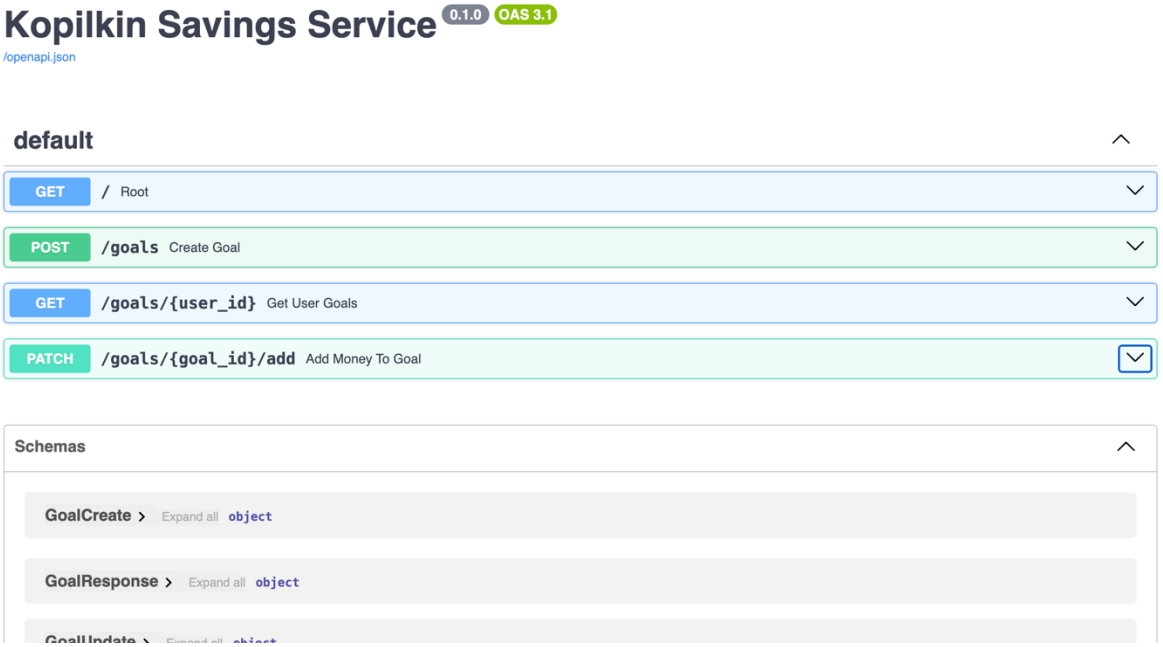
1. **Auth Service**
2. **Transaction Service**
3. **Savings Service**

Каждый из этих сервисов отвечает за собственную предметную область, имеет собственные API endpoints и может запускаться независимо от других компонентов системы.









Screenshot 1-3: три отдельных скрина работающего Swagger UI ***/docs***

4.1. Auth Service

Auth Service отвечает за аутентификацию и базовое управление пользователями.
В рамках минимальной рабочей версии сервис реализует:

- регистрацию пользователя по email и password,
- вход пользователя по email и password,
- получение информации о пользователе по user_id,
- подготовленный endpoint для будущей интеграции входа через Google account.

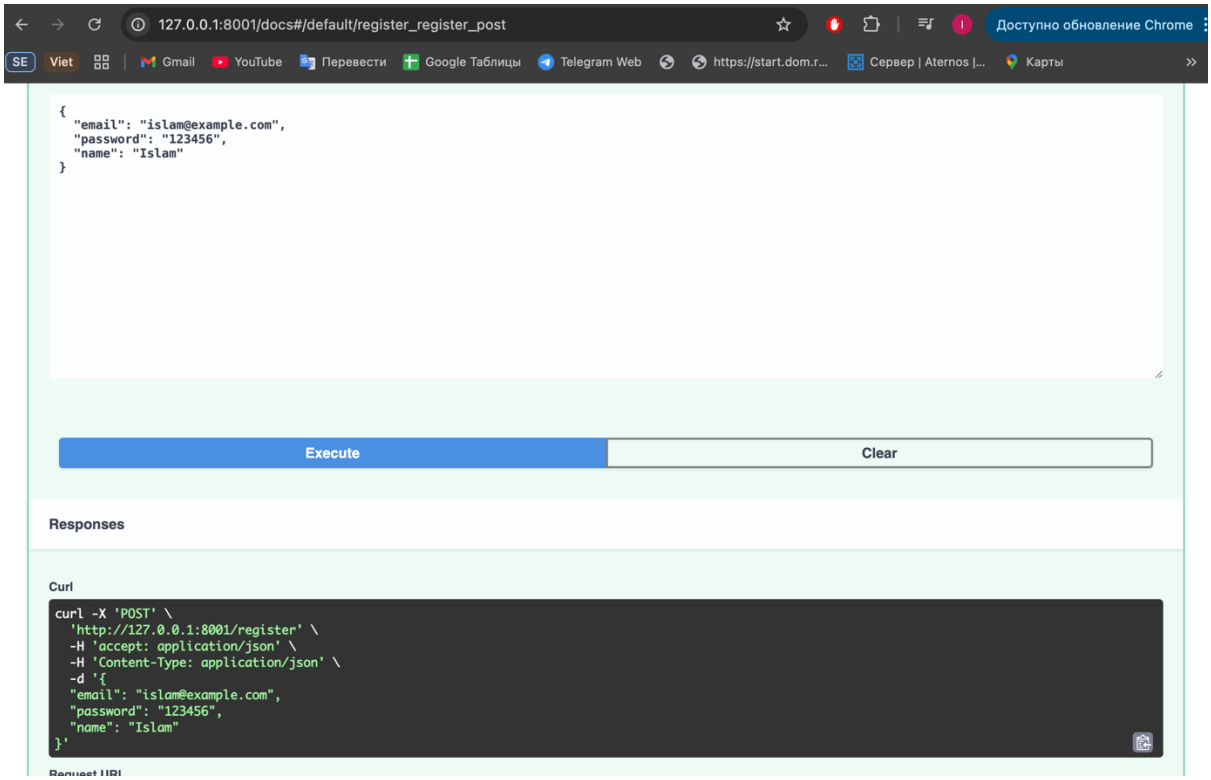
Основная задача этого сервиса — изолировать всю логику, связанную с идентификацией пользователя, в отдельный модуль системы. Это делает архитектуру более чистой и упрощает дальнейшее расширение, например для OAuth 2.0 или token-based authorization.

Auth Service responsibilities:

- user registration,
- user login,
- basic user retrieval,
- future Google OAuth integration.

Endpoints:

- POST /register
- POST /login
- GET /users/{user_id}
- GET /google/login



*Screenshot 4: успешный вызов **/register***

*Screenshot 5: успешный вызов **/login***

4.2. Transaction Service

Transaction Service отвечает за работу с пользовательскими транзакциями и расходами.
В MVP он реализует:

- создание транзакции,
- получение списка транзакций пользователя,

- получение краткой summary по доходам, расходам и категориям.

Каждая транзакция в текущей версии содержит:

- user_id,
- amount,
- category,
- date,
- type (expense или income),
- description (необязательное поле).

Этот сервис играет ключевую роль в проекте, поскольку именно через него пользователь фиксирует свои повседневные финансовые действия. На основе этих данных в дальнейшем могут строиться аналитика, рекомендации и автоматические правила накопления.

Transaction Service responsibilities:

- manual transaction creation,
- storage and retrieval of user transactions,
- category-based aggregation,
- summary generation.

Endpoints:

- POST /transactions
- GET /transactions/{user_id}
- GET /transactions/{user_id}/summary

*Screenshot 6: успешное создание транзакции через **POST /transactions***

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8002/transactions/69fd2440-3b4b-4718-aa8c-6b9be1848aa3/summary' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8002/transactions/69fd2440-3b4b-4718-aa8c-6b9be1848aa3/summary
```

Server response

Code	Details
200	Response body


```
{
  "user_id": "69fd2440-3b4b-4718-aa8c-6b9be1848aa3",
  "total_expense": 2997,
  "total_income": 0,
  "by_category": {
    "food": 2997
  }
}
```


Download

Response headers

```
content-length: 120
content-type: application/json
date: Tue,17 Mar 2026 23:34:05 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Screenshot 7: результат GET /transactions/{user_id}/summary

4.3. Savings Service

Savings Service отвечает за накопительные цели пользователя.
В текущей версии сервис позволяет:

- создать новую цель накопления,
- получить список целей пользователя,
- добавить определённую сумму к выбранной цели,
- автоматически пересчитать процент прогресса.

Каждая цель содержит:

- user_id,
- title,
- target_amount,
- current_amount,
- progress_percent.

Выделение накопительных целей в отдельный сервис обосновано тем, что управление целями и прогрессом является самостоятельной частью предметной области проекта и в будущем может быть расширено правилами накопления, автоматическими рекомендациями и связью с транзакционными событиями.

Savings Service responsibilities:

- savings goal creation,
- savings goal retrieval,
- updating current amount,
- calculating progress percentage.

Endpoints:

- POST /goals
- GET /goals/{user_id}
- PATCH /goals/{goal_id}/add

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8003/goals' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "user_id": "69fd2440-3b4b-4718-aa8c-6b9be1848aa3",
  "title": "Vacation",
  "target_amount": 10000
}'
```

Request URL

```
http://127.0.0.1:8003/goals
```

Server response

Code	Details
200	Response body

```
{
  "id": "3424e2dd-3e75-4770-8292-fd2266eb63d0",
  "user_id": "69fd2440-3b4b-4718-aa8c-6b9be1848aa3",
  "title": "Vacation",
  "target_amount": 10000,
  "current_amount": 0,
  "progress_percent": 0
}
```

Download

Response headers

```
content-length: 181
content-type: application/json
date: Tue,17 Mar 2026 23:35:02 GMT
server: uvicorn
```

Responses

Code	Description	Links

*Screenshot 8: успешное создание цели через **POST /goals***

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8003/goals/3424e2dd-3e75-4770-8292-fd2266eb63d0/add' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "amount_to_add": 1000
}'
```

Request URL

```
http://127.0.0.1:8003/goals/3424e2dd-3e75-4770-8292-fd2266eb63d0/add
```

Server response

Code	Details
200	Response body

```
{
  "id": "3424e2dd-3e75-4770-8292-fd2266eb63d0",
  "user_id": "69fd2440-3b4b-4718-aa8c-6b9be1848aa3",
  "title": "Vacation",
  "target_amount": 10000,
  "current_amount": 1200,
  "progress_percent": 12
}
```

Download

Response headers

```
content-length: 185
content-type: application/json
date: Tue,17 Mar 2026 23:36:09 GMT
server: uvicorn
```

Responses

Code	Description	Links

*Screenshot 9: успешное добавление суммы к цели через **PATCH /goals/{goal_id}/add***

4.4. Why three separate services were chosen

Три отдельных сервиса были выбраны не случайно. Такое разделение отражает три основные функциональные зоны проекта:

- **аутентификация и идентификация пользователя**,
- **управление расходами и транзакциями**,
- **управление накопительными целями**.

Это решение делает систему:

- более понятной,
- более масштабируемой,
- более удобной для дальнейшего сопровождения,
- ближе к реальной микросервисной архитектуре.

Кроме того, такое разделение облегчает дальнейшее развитие проекта в следующих task blocks, где могут быть добавлены event-driven communication, caching, external integrations и routing components.

4.5. Interaction between services

На текущем этапе сервисы логически связаны через общий пользовательский сценарий. Например:

1. Пользователь регистрируется или входит через **Auth Service**.
2. Используя свой user_id, пользователь создаёт транзакции в **Transaction Service**.
3. С тем же user_id пользователь создаёт накопительные цели в **Savings Service**.
4. Данные из **Transaction Service** могут автоматически влиять на прогресс целей в **Savings Service**.

Таким образом, даже в минимальной версии уже прослеживается реальная сервисная декомпозиция и будущая возможность расширения архитектуры.

EN

Microservice Architecture Description

For the first version of **Kopilkin**, a microservice architecture was chosen. This approach matches the assignment requirement to provide at least three microservices and also allows the system to be logically divided into independent functional parts. In the current MVP, three core services were implemented:

1. **Auth Service**
2. **Transaction Service**
3. **Savings Service**

Each of these services is responsible for its own domain area, has its own API endpoints, and can run independently of the other system components.

<u>Screenshots are inserted above</u>

4.1. Auth Service

The **Auth Service** is responsible for authentication and basic user management. In the minimal working version, the service provides:

- user registration via email and password,
- user login via email and password,
- user retrieval by user_id,
- a prepared endpoint for future Google account sign-in integration.

The main purpose of this service is to isolate all user identification logic in a separate module of the system. This makes the architecture cleaner and simplifies future extension, for example for OAuth 2.0 or token-based authorization.

Auth Service responsibilities:

- user registration,
- user login,
- basic user retrieval,
- future Google OAuth integration.

Endpoints:

- POST /register
- POST /login
- GET /users/{user_id}
- GET /google/login

<u>Screenshots are inserted above</u>

4.2. Transaction Service

The **Transaction Service** is responsible for user transactions and expense handling.
In the MVP, it supports:

- transaction creation,
- retrieval of the user's transaction list,
- a short summary of income, expenses, and category totals.

Each transaction in the current version contains:

- user_id,
- amount,
- category,
- date,
- type (expense or income),
- description (optional field).

This service plays a key role in the project because it is the component through which the user records everyday financial actions. Based on this data, analytics, recommendations, and automatic saving rules can later be built.

Transaction Service responsibilities:

- manual transaction creation,
- storage and retrieval of user transactions,
- category-based aggregation,
- summary generation.

Endpoints:

- POST /transactions
- GET /transactions/{user_id}
- GET /transactions/{user_id}/summary

<u>Screenshots are inserted above</u>

4.3. Savings Service

The **Savings Service** is responsible for the user's savings goals.
In the current version, the service allows the user to:

- create a new savings goal,
- retrieve the list of savings goals,
- add a certain amount to a selected goal,
- automatically recalculate the progress percentage.

Each goal contains:

- user_id,
- title,
- target_amount,
- current_amount,
- progress_percent.

Separating savings goals into a dedicated service is justified because goal management and progress tracking represent a standalone domain area of the project, which may later be extended with saving rules, automatic recommendations, and links to transaction events.

Savings Service responsibilities:

- savings goal creation,
- savings goal retrieval,
- updating current amount,
- calculating progress percentage.

Endpoints:

- POST /goals
- GET /goals/{user_id}
- PATCH /goals/{goal_id}/add

<u>Screenshots are inserted above</u>

4.4. Why three separate services were chosen

The three separate services were not chosen randomly. This decomposition reflects the three core functional areas of the project:

- **user authentication and identification**,
- **expense and transaction management**,
- **savings goal management**.

This decision makes the system:

- easier to understand,
- more scalable,
- easier to maintain,
- closer to a realistic microservice architecture.

In addition, such decomposition simplifies future system evolution in later task blocks, where event-driven communication, caching, external integrations, and routing components may be introduced.

4.5. Interaction between services

At the current stage, the services are logically connected through a shared user scenario. For example:

1. The user registers or logs in through the **Auth Service**.
2. Using their user_id, the user creates transactions in the **Transaction Service**.

3. Using the same user_id, the user creates savings goals in the **Savings Service**.
4. Data from the **Transaction Service** may automatically affect progress in the **Savings Service**.

Thus, even in the minimal version, the system already demonstrates real service decomposition and future architectural extensibility.

RU

В текущей версии проекта реализован **minimal microservice prototype**. Система уже разделена на три независимых сервиса по предметным областям, каждый из которых запускается отдельно и имеет собственные API. При этом архитектура пока остаётся упрощённой по сравнению с production-grade микросервисами, поскольку data layer, broker-based communication, caching и routing layer планируются для следующих task blocks.

Сейчас:

- users хранятся в памяти auth-service
- transactions в памяти transaction-service
- goals в памяти savings-service
- текущий пользователь на фронте в localStorage

Потом:

- всё это лучше перенести в **PostgreSQL**
- логически данные останутся разделены по сервисам
- позже можно добавить Redis для cache

EN

The current version of the project implements a minimal microservice prototype.
The system is already divided into three independent services by subject area, each of which runs separately and has its own API. However, the architecture remains simplified compared to production-grade microservices, as the data layer, broker-based communication, caching, and routing layers are planned for future task blocks.

Currently:

- Users are stored in-memory in the auth-service
- Transactions in-memory in the transaction-service
- Goals in-memory in the savings-service
- The current user on the front end is in localStorage

Later:

- It's best to move all of this to PostgreSQL
- Data will remain logically separated by services
- Redis can be added for caching later